

Phase 2 – Project Report
CMPT 276 – Intro to Software Engineering
Escape from the Burnaby Mountain Prison

Group Members:

Jagdeep Sidhu
Arsh Behniwal
Taranjot Aujla
Dominic Janus
Nick Zhou

March 10, 2026
Simon Fraser University

Contents

1	Overall Approach	3
2	Design Adjustments	4
3	Management Process	4
4	External Libraries	6
5	Challenges	6

1 Overall Approach

Our game “Escape from the Burnaby Mountain Prison” is a tile-based 2D prison escape game that is implemented entirely in Java, the goal of the game is to have the player navigate through the map, collect rewards, and reach the exit all while avoiding guards and hazards scattered randomly around the level. While the entire game is built from scratch, we drew heavy inspiration from “The Escapist”, a well known prison escape game where the player has to tread carefully to avoid the guards and strategically plan their escape. To ensure that our project felt fresh and exciting, we decided to emphasize the tension of navigating through the map while under the constant pressure from guards actively pursuing the player.

We had decided from the start that in order to create a successful group project, we would need a way to organize our code so that the entire team would be able to work simultaneously on the project. We thus structured the game around a shared base class for entities that moved on the map, such as the Player and Guard classes, which inherit from a common Entity class that handles the fundamentals of movement and positioning. This turned out to be a great help, as it meant that different team members would be able to work on the Player’s and Guard’s behaviour independently without constantly running into any issues, it also meant that movement that was improved in the base Entity system would benefit both the Guard and Player class.

The player character is implemented as a Player class which extends from the shared Entity base, movement for the player is tile based, and you can control their movement through keyboard inputs. Some things we had to keep in mind while implementing this class was balancing the speed of the character with the visual output, as we wanted the player’s movement to look smooth and natural. Another big thing we had to keep in mind while implementing the Player class was handling core gameplay mechanics, like managing lives, collecting rewards, avoiding guards and hazards, as well as lost conditions like lives or score dropping below zero. Compared to our initial Phase 1 design, the scoring system and life mechanics were adjusted to make the game more engaging and balanced.

The game runs at a fixed 30 frames per second, updating all entity locations and checking interactions each frame before redrawing the screen. The map is built on a grid of tiles, where each tile can be a wall, floor, hazard, reward, powerup, or coin, and there are guards with fixed patrol routes who react to the player when spotted. The map is designed to look and feel like a real environment, with corridors, rooms, and obstacles that the player has to navigate around, the guards included in the map are equipped with pathfinding to ensure that they’re able to navigate around the walls and smoothly follow the player’s movements. While the player is able to pick up powerups that grant them temporary advantages like freezing the guards, or increased movement speed, there are also multiple hazards that are scattered throughout the map that penalize the player. These hazards ensure that the game does not feel too bland, and a coin system was implemented into the game so that it would be mandatory for the player to explore and traverse around the map. The game also supports three difficulty levels, which adjust the number of lives the player starts with and how many guards are on the map, allowing players to push themselves if they clear the previous difficulty.

The guards are an important aspect of the game, and we decided that we would have them patrol a fixed route but would switch to a chase state if the player came within a certain distance and was detected within their line of sight. If the player manages to move far enough

away, the guard abandons the chase and returns to its original patrol route, so that they are not chasing the player for an indefinite period of time. To ensure that the guards behaved in a way that felt both smooth and fair, pathfinding was implemented so that they would be able to navigate around walls and obstacles when chasing the player or returning to their patrol areas. Another thing we kept in mind while designing the guards was their spacing, as we did not want the guards to all spawn in a single corner of the map. We thus decided to split the map into five, six and seven zones depending on the difficulty of the game mode, since five guards spawn in easy mode, six in medium, and seven in hard, this ensured that the player would consistently encounter the guards no matter which paths they decided to take.

2 Design Adjustments

As development progressed, we recognized that our initial Phase 1 design needed several structural modifications to make our game run more smoothly and minimize the amount of duplicate code. Originally, our Player and Guard entities were designed to operate independently. However, as we began implementing the grid-based movement, we realized this approach would result in significant amounts of code duplication. To solve this, we introduced an abstract Entity base class that includes the precise sub-grid coordinates and the collision-detection logic used to navigate the map tiles. This adjustment allowed for us to refine the movement physics in one central location while also making the code much cleaner and easier to maintain.

On the topic of the movement system, we also had to completely rethink our enemy tracking mechanics. Our Phase 1 design assumed guards would track the player using simple directional logic along the X and Y axes. During early testing, we discovered this caused guards to get stuck along walls when attempting to navigate around corners. Thus, we had to redesign our guard mechanics, and we implemented Breadth-First Search (BFS) for pathfinding alongside a custom line-of-sight algorithm, which allows the guards to intelligently navigate the map and fixed the issue of them getting stuck on walls. To ensure this more complex implementation for the guard logic did not negatively impact the game's performance, we implemented a caching system where the guard's path is only recalculated every 5 frames.

Furthermore, to ensure the game performed consistently, we adjusted the core engine's execution speed. We initially planned to run the primary game loop at a standard 60 frames per second; however, we found this would make the game run way too fast and also create unexpected lag spikes. Therefore, we made the design decision to drop the frame rate to a locked 30 frames per second. This adjustment gave the game a much smoother, arcade-like feel that perfectly matched the aesthetic we were going after. In addition, locking the game to 30 frames per second provided the engine with significantly more processing overhead, which helped ensure our new BFS pathfinding and continuous collision-detection loops could execute reliably without stuttering or dropping frames.

3 Management Process

Our management process was loosely inspired by the agile scrum software development lifecycle, where work is divided into smaller tasks and completed incrementally. Weekly meetings

functioned similarly to sprint planning sessions, allowing the team to review progress, assign new tasks, and adapt to new goals. We held weekly in-person meetings on Fridays to review the current state of the game, discuss future development goals, and address any challenges we faced. Throughout the week, we communicated through a Discord group chat to share progress updates, coordinate smaller tasks and ask for help. At the beginning of phase 2, we distributed roles based on the core functions of the game. These roles gave each of us a specific focus and avoided any possible merge conflicts and overlapping. Continuing, during our weekly meetings, we created a list of smaller functions to be implemented and other tasks to work on and assigned each to a group member to work on.

Roles & Responsibilities:

- **Arsh Behniwal:** Focused on the core game loop thread, continuous input handling, graphics rendering, the storyline screen, the reward system, and the audio system. Implemented the 30 FPS game-loop to ensure smooth game play with a classic arcade feel, handled keyboard input processing, and developed the win/ lose logical conditions and reward collection logic. Additionally, designed graphical sprites representing entities, objects, and map elements, and managed the audio system, ensuring background music and sound effects were triggered at the appropriate times/ events.
- **Jagdeep Sidhu:** Focused on implementing the game objects, such as the power-ups and static enemies. Responsible for spawning the correct amount of each object onto the correct tile. Programmed unique power-up effects such as the coffee speed boost, doctor's note life restoration, and Snow Day power-up, which temporarily freezes guard movement. Additionally, implemented the unique hazard effects, including point reduction from colliding with a bear, handcuffs, and inverted player movement caused by spoiled milk. Developed the scoring system that adds or deducts points based on the player's interaction with these objects.
- **Taranjot Aujla:** Managed the construction and initialization of the PrisonMap and MapPanel classes. This included developing the 2D map grid, managing tile types for collision detection and player movement, and establishing the map layout by setting the start and end tiles. Additionally, implemented the randomized spawn logic to distribute the coins across valid tiles at start-up.
- **Nick Zhou:** Responsible for developing the guard mechanics. Engineered the core AI player tracking behaviour using breadth-first-search. Assigning each guard to a distinct patrol zone across the map and initializing a specific number of guards based on the difficulty. Moreover, implemented the collision detection mechanics, ensuring that when a guard interacts with the player, a life is deducted and the damage is visually indicated.
- **Dominic Janus:** Developed the player interaction and visual rendering systems. Involved programming player movement, managing the loading and drawing of all sprite assets, and designing the jail cell map graphics. Additionally, implemented the complete user interface. This included the running HUD and interactive start, pause, and end menus. Also created a dynamic 'fog of war' feature to restrict map visibility and increase game immersion.

We set our midway checkpoint as February 25th to match our check-in meeting with the T.A. Our goal was to have a functioning game with the map rendering on the screen, basic player movement and object collision, and static enemies. We successfully met these goals, which provided a foundation for implementing more advanced and polished game play features in the second half. We planned on having the game fully completed by March 9th, and we also met this deadline for the majority of the game. Some finishing touches and small errors were left until the next day.

Using this management process allowed the team to work in parallel while minimizing merge conflicts and overlapping work. We found the most effective aspect of our workflow was meeting in person and creating a list of functions and design implementations needed for the game. Regular communication and clearly defined responsibilities ensured that progress remained consistent throughout the coding phase

4 External Libraries

For this project, we did not rely on any major third-party gameplay libraries or external game engines. Instead, we chose to build the game using Java's built-in libraries. This gave us more control over the mechanics and also helped us understand how each part of the game worked behind the scenes. The main library we used was `javax.swing`. It allowed us to create the game window, menus, panels, and the timer-based update system. This was especially helpful because we could build the title screen, pause menu, and main game panel directly in Java without needing to learn a separate framework. We also used `java.awt` and `java.awt.image` for the graphics side of the project. These libraries handled things like drawing the map, rendering sprites, displaying the HUD, and creating visual effects such as the fog-of-war system.

For assets, we used `javax.imageio.ImageIO` to load sprite files from our resources folder. This made it easy to organize and display our custom textures for characters, objects, and map tiles. We also used `javax.sound.sampled` for audio, including background music and sound effects for actions like collecting rewards, taking damage, winning, and losing. This made the game feel more polished and responsive. Finally, while it is not a game play library, we used Maven's `maven-jar-plugin` to package the project into a runnable `.jar` file. This made the finished game easier to test and distribute. Overall, Java's standard libraries were more than enough for a project of this size, and using them let us focus on building the systems ourselves rather than depending on an external engine.

5 Challenges

- **Nick Zhou:** The most significant challenge I experienced was implementing the guard AI, as my initial approach used simple directional movement towards the player, and after a bit of testing I discovered that there were two major flaws with this line of thinking. Guards would get stuck on walls when the player was around a corner, and the guards would be able to detect the players through the walls due to its distance only aggression check. To dissolve this issue, I did some research and discovered that I would have to implement BFS pathfinding, however, running BFS every frame for every guard introduced some performance issues, and it had to be fixed by caching the computed

step direction and only recalculating the path every five frames. Regarding the problem with the guard's line of sight, I watched a couple videos and went through some forums and discovered that it could be solved by using Bresenham's algorithm, which steps through the tiles between the guard and the player and returns false if it encounters an unwalkable tile. This makes it so that the guard only starts chasing when they have a clear view of the player, which makes the AI a lot more natural and pristine.

- **Arsh Behniwal:** The most significant challenge I faced was integrating the audio with the `javax.sound` API. I initially encountered a `LineUnavailableException`. After further research, I discovered that some of the audio files we chose were not in the correct 16-bit PCM format as required by Java's Clip system. To resolve this, I had to manually re-encode the `.wav` files using an external audio software, Audacity, so the Java engine could successfully parse them without throwing exceptions.
- **Dominic Janus:** The biggest challenge I dealt with was improving the visual presentation of the game while still keeping everything functional and readable during gameplay. Once the player, guards, hazards, rewards, and menus were all on screen together, it became clear that visibility and interface design were going to be a major issue. Initially, the map felt too zoomed out, the player could see too much of the level at once, and several HUD elements were difficult to read against the background. Another challenge was making sure that all of the sprite assets loaded properly and were rendered in the correct places, especially as we added more visual states. To address this, I worked on the camera system so that it followed and centered on the player, added a fog-of-war effect, and redesigned the HUD so that information like lives, score, rewards, and time were easier to see. I also had to ensure that sprite loading was handled consistently through the resource system so that the game would still display properly once packaged into a runnable `.jar` file.
- **Jagdeep Sidhu:** My primary challenges revolved around state management and complex entity interactions. Firstly, implementing temporary status effects (such as the Parking Ticket speed decrease or the Renaissance Coffee speed boost) required tightly syncing multiple timers to the core 30 frames per second game loop. Since the movement physics rely directly on the engine's tick rate, I had to manipulate the player's speed variable and ensure it reset once the effect timers expired. Secondly, handling edge-case collisions required building restrictive logic across multiple files. For example, when implementing the Handcuffs static hazard, I had to engineer checks between multiple classes to ensure that a tied-up player was physically blocked from collecting any powerups or rewards (required a lot of testing and fixing) until the hazard penalty cleared.
- **Tarnjot Aujla:** My biggest challenge was managing the 2D map grid and the randomized coin spawning. I had to carefully configure the floor and wall tiles to ensure the game's collision detection worked flawlessly. Additionally, writing the spawn algorithm was tricky; I had to make sure the coins only scattered across valid, walkable tiles and never accidentally spawned inside walls or outside the map boundaries.